

 **fdtn-ai/antares-350m** 

♡ like 67

Follow  Cisco Foundation AI 870

 Text Generation  Transformers  Safetensors  English granitemoehybrid security

vulnerability-detection agentic terminal-agent reinforcement-learning granite conversational

 License: apache-2.0



 Deploy ▾

→ Copy to bucket **NEW**

Use this model ▾



Model card

Files

 xet



Community



You need to agree to share your contact information to access this model

This repository is publicly accessible, but **you have to accept the conditions to access its files and content.**

Please complete the form below to request access to this model. All requests are reviewed manually.

By submitting this form, you confirm that the information provided is accurate, agree to share your contact details with the repository authors. You will be subject to their communications and Privacy Statement:

<https://www.cisco.com/c/en/us/about/legal/privacy-full.html>

Log in

or

Sign Up

to review the conditions and access this model content.

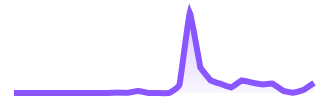
Antares-350M

Antares-350M is an open-weight, 350-million parameter language model specialized for vulnerability localization in real-world codebases. Built on IBM Granite 4.0 350M, it is trained to autonomously navigate source code repositories

Downloads last

month

1,760



 **Safetensors** ⓘ

Model size 0.4B params

Tensor type BF16

 Chat template

↗ Files info

 **Inference Providers** **NEW**

 Text Generation

This model isn't deployed by any Inference Provider.



Ask for provider support

 **Model tree for fdtn-ai/antares-3...**

Base model [ibm-granite/granite-4.0...](#)

 **Finetuned** [ibm-granite/granit...](#)

 **Finetuned (19)** ... [this model](#)

Finetunes 1 model

Quantizations ⚡ 📄 🖱 8 models

through a terminal interface — issuing shell commands, reading files, and iteratively narrowing in on vulnerable files — using a two-stage pipeline: supervised fine-tuning (SFT) followed by Group Relative Policy Optimization (GRPO).

Unlike traditional static analysis tools or retrieval-based approaches, Antares-350M operates as a terminal agent: it explores repositories by executing commands (`grep`, `find`, `cat`, and standard Unix utilities), reasons about the output, and submits a list of files it believes to contain the reported vulnerability.

Antares-350M is part of the Antares family of models (350M, 1B, 3B), all built on IBM Granite 4.0 and trained with the same two-stage pipeline. The 350M model achieves a File F1 of **0.135** on the Vulnerability Localization benchmark (VLoc Bench), outperforming models many times its size including Gemini 2.5 Flash, Gemma-4-31B, GPT-5 Mini, and Qwen3.5-122B-A10B.

Within the family, Antares-350M can serve as a speculative draft model for the larger Antares variants in compatible deployments: it proposes candidate tokens that a 1B or 3B model verifies, accelerating inference while preserving the larger model's localization quality. It can also run standalone in resource-constrained deployments.

Antares enables organizations to deploy compact, on-premises vulnerability localization capabilities without relying on cloud-based AI services. In our evaluation setup, Antares-350M completes the full 500-task VLoc Bench sweep in approximately 11

Collection including fdtn-ai/anta...



Antares



Collection

Agentic Mo... • 2 items • Upd... • 56

minutes on a single H100 GPU using 16 parallel workers.

- **Model Name:** Antares-350M
- **Model Developer:** Copyright © 2026 by Cisco Systems, Inc. All rights reserved. Cisco product group: Foundation AI
- **Model Card Contact:** <https://fdtn.ai/contact>
- **Technical Report:** Vijay, Priyanshu, et al. "Antares: Foundation Models for Agentic Vulnerability Localization." Foundation AI, Cisco, 2026.
- **Model Release Date:** 2026
- **Supported Language(s):** English
- **Model Architecture:** Auto-regressive decoder-only transformer (IBM Granite 4.0 350M backbone): 28 layers, hidden dim 1024, 16 attention heads, 4 KV heads (GQA), 32K context window, 100,352 vocab, SwiGLU activation, RMSNorm, RoPE positional encoding.
- **Training Objective:** Two-stage pipeline: SFT on deep-research, terminal-navigation, and cybersecurity-reasoning data, followed by GRPO with multi-component verifiable rewards over complete multi-turn agent trajectories, including file-level localization quality, valid submission behavior, tool-use compliance, and exploration behavior.
- **Training Data Status:** This is a static model trained on an offline dataset with a data cutoff of April 10, 2025. Future versions will be released on updated data.
- **License:** Apache 2.0

Intended Use

Intended Use Cases

Antares-350M is designed for security practitioners, researchers, and developers building automated vulnerability-localization workflows. The model is optimized for:

- **Vulnerability Localization:** Given a CWE identifier, its generic category description, and a repository, identifying which source files contain the reported vulnerability — using only a terminal interface, without any external retrieval system or vector database.
- **Shift-Left Security:** Integrating into CI/CD pipelines to surface vulnerable files early in the development lifecycle. The 350M model runs on a single GPU, making it practical for automated pipeline use.
- **Advisory-Driven Triage:** Using CWE identifiers and generic category descriptions, including those mapped from CVE or GHSA records, to guide targeted, multi-step repository exploration that narrows down candidate files for analyst review.

The model is intended for local deployment in environments prioritizing data security, regulatory compliance, and operational control.

Downstream Use

Antares-350M can be integrated into security toolchains as a terminal-based agentic component. Example downstream applications include:

- **Vulnerability Search:** Locating vulnerable files from CWE identifiers and generic category descriptions by issuing shell commands against a repository snapshot
- **Security Workflow Integration:** Augmenting static analysis tools with reasoning-driven repository exploration; using CWE mappings derived from CVE or GHSA records to surface relevant code; supporting security analysts in evidence gathering and hypothesis refinement
- **Automated Code Triage:** Iteratively narrowing down candidate files using grep, find, and content inspection to surface security-relevant code for analyst review

For questions or assistance with integrating Antares-350M, please contact Hadas Birin (hbirin@cisco.com).

Out-of-Scope Use

The following uses are out-of-scope and are neither recommended nor intended use cases:

1. **Generating harmful content** — The model should not be used to:
 - Generate malware or other malicious code
 - Create phishing content or social engineering scripts
 - Develop attack plans targeting specific organizations
 - Design exploitation techniques for vulnerabilities without legitimate security research purposes

2. **Critical security decisions without human oversight** — The model should not be used for:

- Autonomous security decision-making without human review
- Critical infrastructure protection without expert supervision
- Final determination of security compliance without human verification
- Autonomous vulnerability remediation without testing

3. **Legal or medical advice** — The model is not qualified to provide legal advice regarding security regulations, compliance requirements, or intellectual property disputes, or medical advice regarding health impacts of security incidents.

4. **General-purpose chat or instruction following** — The model is specifically trained for terminal-based vulnerability localization and may not perform well on general conversational or instruction-following tasks.

5. **Standalone safety evaluation** — This model is designed to operate within an agentic terminal loop rather than as a general-purpose chat assistant. Standard safety benchmarks such as HarmBench evaluate direct conversational refusal behavior, which does not reflect how this model is intended to be deployed. Safety considerations should instead be addressed at the system level through access controls, sandbox isolation, and human oversight of model outputs.

6. **Violation of Laws or Regulations** — Any use that violates applicable laws or regulations.

How to Get Started with the Model

Antares-350M operates as a terminal agent: it receives a system prompt containing a CWE identifier and its generic category description, then iteratively generates reasoning and tool calls whose outputs are appended to the conversation context. The model may issue up to 15 terminal calls and must then terminate through either `submit_vulnerable_files` or `submit_no_vulnerability_found`.

```
from transformers import AutoTokenizer, AutoModelForCausalLM
```

```
tokenizer = AutoTokenizer.from_pretrained("facebook/antares-350m")
model = AutoModelForCausalLM.from_pretrained("facebook/antares-350m")
```

```
# The model uses a structured tool-calling format
#   <think> ... </think>                - Reasoning
#   <tool_call> {"name": "terminal", "arguments": {}} - Tool call
#   <tool_response> ... </tool_response>    - Tool output
#
# The agent loop terminates when the model calls
#   submit_vulnerable_files - list of file paths to check
#   submit_no_vulnerability_found - declares no vulnerability found
```

```
SYSTEM_PROMPT = """You are a security vulnerability scanner.
You have access to a terminal with the repository files.
Use shell commands (grep, find, cat, etc.) to search for
files that contain the reported vulnerability description.
```

```
cwe_description = (
    "CWE-78: Improper Neutralization of Special Characters in Untrusted "
    "Input Data in Command Lines ('OS Command Injection'). The software "
    "uses untrusted input without properly neutralizing it, "
    "using externally-influenced input from an upstream component to "
    "control the flow of execution or invoke unauthorized operations or "
    "neutralize or incorrectly neutralizes special characters before "
    "executing any untrusted input."
```

```

        "intended OS command when it is sent to :
    )

messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": f"Vulnerability: {vulnerability}"},
]

# Apply chat template and generate the first
inputs = tokenizer.apply_chat_template(
    messages, return_tensors="pt", add_generation_prompt=True
)
prompt_length = inputs["input_ids"].shape[-1]
output = model.generate(
    **inputs,
    do_sample=True,
    temperature=0.3,
    top_p=1.0,
    max_new_tokens=512,
)
response = tokenizer.decode(output[0][prompt_length:])
print(response)

# The model will output a <think> block followed by a <tool_call> block
# Your infrastructure should parse the tool call and execute the command
# and append the result as a <tool_response> block
# Repeat until the model calls submit_vulnerability_report

```

The model is designed to operate within an agentic evaluation loop with a sandboxed terminal environment. Each repository should be extracted into an isolated container (e.g., ubuntu:24.04 with network disabled) before the agent begins exploration. See the technical report for full integration details and the recommended agent loop implementation.

Training and Evaluation

Training Data

Antares-350M was trained through a two-stage pipeline:

Stage 1 — Supervised Fine-Tuning (SFT):

A proprietary corpus spanning three categories: (1) cybersecurity reasoning data covering vulnerability concepts, CWE/CVE reasoning, threat modeling, advisory interpretation, and security analysis; (2) deep-research and general reasoning data focused on multi-step reasoning, evidence aggregation, and instruction following; and (3) code-search trajectories demonstrating terminal-based repository exploration, file inspection, and iterative search behavior.

Stage 2 — GRPO (Group Relative Policy

Optimization): Reinforcement learning over complete multi-turn agent trajectories using vulnerable repository snapshots curated through proprietary data-generation and filtering pipelines. Each task includes a complete codebase containing a known vulnerability and ground-truth labels identifying the affected implementation files.

Data cutoff: April 10, 2025.

This is a static model trained on an offline dataset. Future versions will be released on updated data. A detailed description of the methodology is available in the technical report.

Training Setup

Antares-350M is based on the **IBM Granite 4.0 350M** architecture. Training was performed on Cisco

Foundation AI's internal compute cluster.

Key training details:

- **Two-stage training:** SFT → GRPO
- **Optimizer:** AdamW
- **Compute:** 8×H100 GPUs

A more detailed description of the methodology is available in the technical report.

Evaluation

Antares-350M was evaluated on the Vulnerability Localization Benchmark (VLoc Bench), which comprises 500 tasks drawn from 290 unique real-world repositories, spans 6 package ecosystems and 147 unique CWE categories, and includes assigned CVE identifiers for 78% of tasks. Each task pairs a vulnerable repository snapshot with ground-truth vulnerable implementation files; tests, documentation, and configuration files are excluded from the label set.

The agent receives a CWE identifier and its generic category description — no advisory text, file hints, or severity details — and must explore the repository within a budget of 15 terminal commands per task, followed by one final submission action.

Metric calculation: For each task, submitted file paths are compared with the ground-truth set of vulnerable implementation files. Precision is the fraction of submitted files in the ground-truth set, recall is the fraction of ground-truth files submitted, and File F1 is the harmonic mean of

task-level precision and recall. A task with no valid file submission receives zero precision, recall, and File F1 because every Phase A task contains a known vulnerability. We macro-average precision, recall, and File F1 across all 500 tasks, then average the resulting metrics across three independent runs. The reported File F1 is therefore the mean task-level File F1 across all tasks and runs.

All evaluations use temperature=0.3 and top_p=1.0.

Overall Performance (Phase A — File F1):

Model	Parameters	File F1 ↑	Precision	Recall
GPT-5.5 (xhigh)	Frontier	0.229	0.310	0.221
Antares-3B (GRPO)	3B	0.223	0.303	0.221
GPT-5.5	Frontier	0.221	0.305	0.211
Antares-1B (GRPO)	1B	0.209	0.262	0.224
Antares-3B (SFT)	3B	0.198	0.240	0.228
Antares-1B (SFT)	1B	0.188	0.263	0.179
GLM-5.2	753B	0.186	0.226	0.186
Gemini 3 Pro	Frontier	0.152	0.190	0.153
Antares-350M (GRPO)	350M	0.135	0.136	0.178

Model	Parameters	File F1 ↑	Precision	Recall
Antares-350M (SFT)	350M	0.108	0.149	0.101
Gemini 2.5 Flash	Frontier	0.102	0.132	0.098
Gemma-4-31B	31B	0.101	0.131	0.097
GPT-5 Mini	Frontier	0.098	0.115	0.096
Gemini 3.1 Flash Lite	Frontier	0.095	0.131	0.090
Qwen3.5-122B-A10B	125B MoE	0.091	0.124	0.083
GPT-5	Frontier	0.048	0.062	0.048
GPT-5 Nano	Frontier	0.024	0.038	0.021
Llama-3.3-70B	70B	0.012	0.016	0.014
Granite 4.0 350M (base)	350M	0.001	—	—

Bold rows indicate Antares models. All models use the same agent protocol, tools, task inputs, 15-call terminal budget, and generation settings of temperature 0.3 and top-p 1.0. Results are averaged across three independent runs.

Safety Alignment

This model is designed to operate as a component within a sandboxed agentic terminal loop, not as a standalone conversational assistant. As such, no additional standalone safety alignment (e.g., RLHF-based refusal training or HarmBench evaluation) was performed on this model.

Safety considerations should be addressed at the system level by:

- Running the agent inside an isolated sandbox environment (e.g., Docker container with `network=none`)
- Implementing access controls on the deployment environment to restrict use to authorized security personnel
- Enforcing human oversight over model outputs before any remediation action is taken
- Monitoring and auditing agent trajectories in high-risk workflows

It is strongly recommended to deploy this model with appropriate system-level safeguards and not to expose it as a general-purpose assistant endpoint.

Limitations

Antares-350M has several limitations that users should be aware of:

1. **Terminal budget constraints:** Performance degrades significantly on large repositories (>10MB) where 15 commands are insufficient to cover the codebase. Multi-file vulnerabilities requiring 5+ files of context also underperform.

2. **Vulnerability pattern dependence:** The model performs best on vulnerability types with distinctive, grep-able code patterns (e.g., CWE-843 Type Confusion, CWE-1321 Prototype Pollution). It performs poorly on CWE types requiring semantic understanding of code behavior rather than pattern matching (e.g., CWE-732 Incorrect Permissions, CWE-667 Improper Locking, CWE-401 Memory Leak).
3. **Knowledge cutoff:** The model may not recognize vulnerabilities, exploit patterns, or attack vectors introduced after its April 10, 2025 training cutoff.
4. **No intended exploitation functionality:** The model is not designed or evaluated to generate proof-of-concept exploits or provide vulnerability explanations. It outputs candidate file paths for human review; its internal reasoning guides the search but is not a substitute for a validated security explanation.
5. **Retrieval system independence:** Unlike embedding-based retrieval approaches, Antares requires no external vector database, but performance is bounded by the quality of its terminal exploration strategy within the allotted command budget.
6. **Potential biases:** The model may reflect biases present in security literature and known attack patterns, and may be biased toward vulnerability types and programming languages more prevalent in its training data.
7. **Security risks:** The model cannot verify the identity or intentions of users. Adversarial

prompting techniques may potentially bypass safety mechanisms if the model is exposed without system-level guardrails.

Recommendations

To address the limitations of Antares-350M, we recommend:

1. **Human oversight:** Always have qualified security professionals review model outputs before taking action. Use the model as an assistive tool rather than a replacement for expert human judgment. Implement a human-in-the-loop approach for security-critical workflows.
2. **Sandbox isolation:** Run the agent inside a Docker container with network disabled (`network=none`), a command timeout (10 seconds recommended), and resource limits (2 CPU cores, 4GB RAM). Destroy the container after each evaluation entry.
3. **System-level access controls:** Deploy the model in environments with appropriate access controls and restrict access to authorized security personnel.
4. **Knowledge supplementation:** Supplement the model with up-to-date security feeds and

 System theme [databases for advisors released after the April 2025 training cutoff.](#)



[Models](#)

[Datasets](#)

[Spaces](#)

[Pricing](#)

[Docs](#)

5. **Usage policies:** Develop and enforce clear acceptable use policies for applications using this model. Implement monitoring and auditing for high-risk applications.